

ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

NT213.P11.ANTT

A18 - PHÁT TRIỂN VÀ BẢO MẬT ỨNG DỤNG ANDROID KOTLIN

Nhóm 11



NỘI DUNG BÁO CÁO

1. THÔNG TIN CHUNG

2. KHAI THÁC VÀ CẢI TIẾN CÁC LỖ HỔNG

3. TỔNG KẾT

1

THÔNG TIN CHUNG

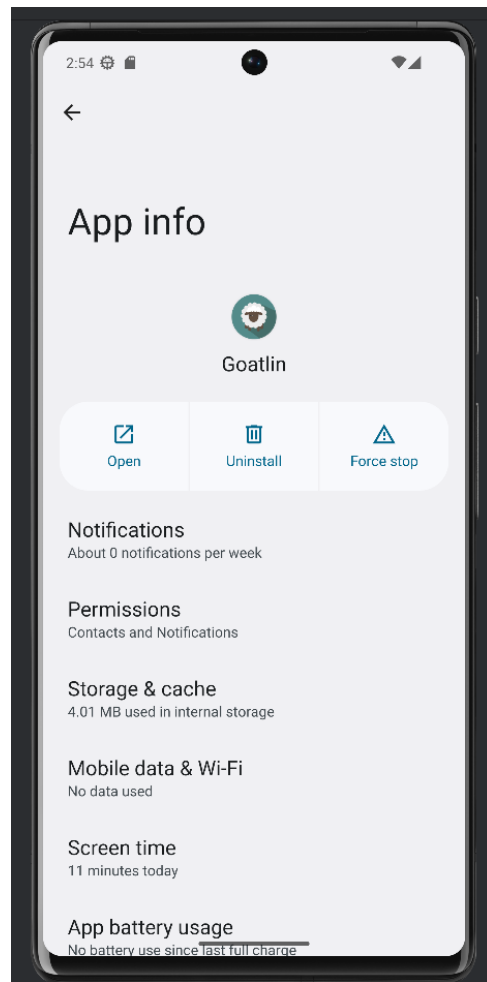
THÔNG TIN CHUNG

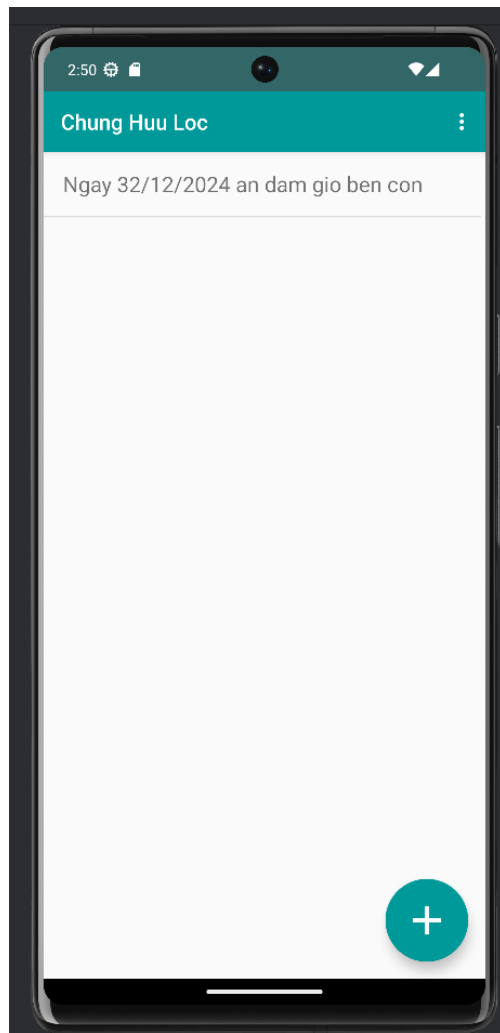
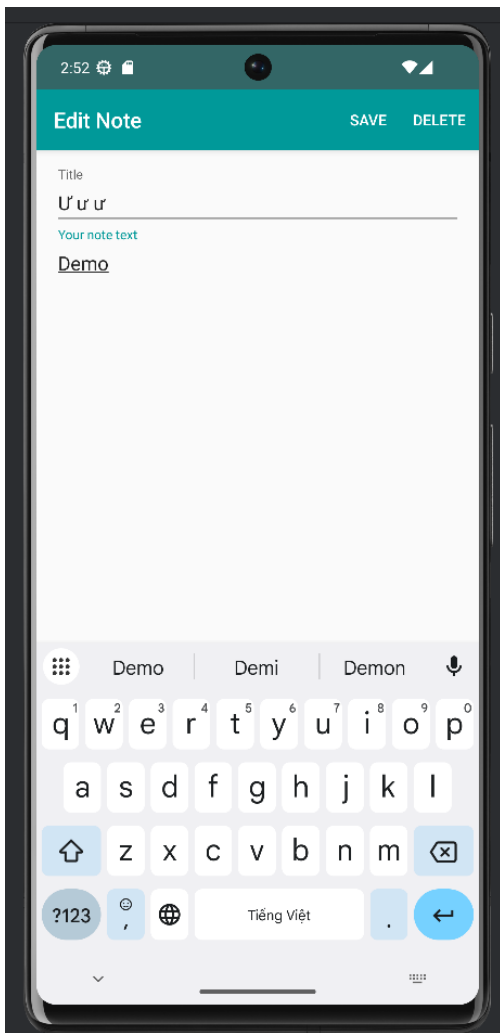
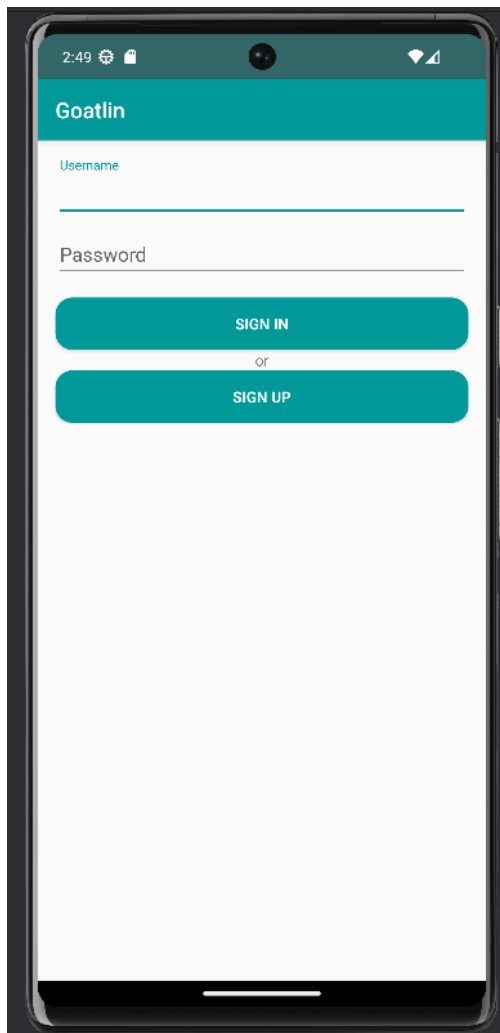
- **Mục tiêu của dự án Goatlin:** Phát hiện, vá các lỗ hổng bảo mật và hoàn thiện một ứng dụng Android bằng ngôn ngữ Kotlin.
- **Nhiệm vụ chính:** Dự án tạo ra một ứng dụng mobile với các lỗ hổng bảo mật cố ý để sinh viên có thể hiểu rõ các rủi ro tiềm ẩn khắc phục chúng
- **Những gì đã đạt được:** gồm những việc đã đề cập ở giữa kỳ và các công việc sau:
 - + Fix được các lỗ hổng trong Top 10 OWASP Mobile 2016
 - + Cải tiến các tính năng bảo mật trong app
 - + Cải tiến được trải nghiệm người dùng UI/UX
 - + Deploy thành công Server với HTTPS và cấu hình Security Rule cho máy chủ

GUI

CẢI TIẾN TRẢI NGHIỆM NGƯỜI DÙNG TRONG APP







2

KHAI THÁC VÀ CẢI TIẾN CÁC LỖ HỔNG

ANDROID KEYSTORE

Android KeyStore là một thành phần bảo mật quan trọng trong hệ điều hành Android, được thiết kế để lưu trữ và quản lý các khóa mã hóa một cách an toàn. Nó bảo vệ các khóa khỏi việc bị đánh cắp hoặc bị lộ bằng cách sử dụng cả biện pháp bảo mật phần mềm và phần cứng (nếu có).



ĐẶC ĐIỂM CHÍNH

Lưu trữ khóa an toàn:

- Không thể bị xuất trực tiếp bởi ứng dụng hoặc hệ thống, chỉ có thể được sử dụng thông qua các API của KeyStore.
- Hạn chế truy cập trái phép vào khóa.

Hỗ trợ phần cứng bảo mật: Trên các thiết bị hỗ trợ **Trusted Execution Environment (TEE)** hoặc **Secure Element (SE)**, Android KeyStore lưu trữ khóa trong một môi trường bảo mật độc lập với hệ điều hành chính.

Bảo mật sinh trắc học: KeyStore tích hợp với các phương thức xác thực sinh trắc học (vân tay, khuôn mặt) để xác thực người dùng trước khi sử dụng khóa.

ĐẶC ĐIỂM CHÍNH

- Lưu trong phần cứng (Hardware-Backed KeyStore):

- + Khóa lưu trong phần cứng bảo mật như **Trusted Execution Environment (TEE)** hoặc **Secure Element (SE)**.
- + Bảo mật cao: Khóa không thể bị trích xuất, bảo vệ trước tấn công vật lý và side-channel.
- + Hiệu năng tốt: Thao tác mã hóa/giải mã thực hiện trực tiếp trên phần cứng.

- Lưu trong phần mềm (Software-Backed KeyStore):

- + Khóa lưu trong bộ nhớ hệ điều hành và được bảo vệ bởi **SELinux**.
- + **Bảo mật thấp hơn**: Dễ bị trích xuất nếu thiết bị bị root hoặc bị tấn công.
- + **Hiệu năng thấp hơn**: Mọi thao tác thực hiện qua phần mềm thay vì phần cứng

PREFERENCEHELPER

Lớp PreferenceHelper là một lớp hỗ trợ lưu trữ và truy xuất dữ liệu trong ứng dụng Android thông qua SharedPreferences.



HÀM SETSTRING

```
// Lưu dữ liệu mã hóa vào SharedPreferences
fun setString(key: String, value: String) {
    val cipher = Cipher.getInstance(TRANSFORMATION)
    cipher.init(Cipher.ENCRYPT_MODE, getSecretKey())
    val iv = cipher.iv // Initialization Vector (IV)
    val encryptedBytes = cipher.doFinal(value.toByteArray())

    // Lưu IV và dữ liệu mã hóa dưới dạng chuỗi
    val ivBase64 = Base64.encodeToString(iv, Base64.DEFAULT)
    val encryptedBase64 = Base64.encodeToString(encryptedBytes, Base64.DEFAULT)
    val combinedValue = "$ivBase64:$encryptedBase64"

    sharedPreferences.edit().putString(key, combinedValue).apply()
}
```

HÀM GETSTRING

```
// Đọc và giải mã dữ liệu từ SharedPreferences
fun getString(key: String, default: String? = null): String {
    val encryptedData = sharedPreferences.getString(key, null) ?: return default ?: ""
    val parts = encryptedData.split( ...delimiters: ".")
    if (parts.size  $\neq$  2) return default ?: ""

    val iv = Base64.decode(parts[0], Base64.DEFAULT)
    val encryptedBytes = Base64.decode(parts[1], Base64.DEFAULT)

    try {
        val cipher = Cipher.getInstance(TRANSFORMATION)
        val spec = GCMParameterSpec( tLen: 128, iv)
        cipher.init(Cipher.DECRYPT_MODE, getSecretKey(), spec)
        return String(cipher.doFinal(encryptedBytes))
    } catch (e: Exception) {
        e.printStackTrace() // Ghi log để kiểm tra lỗi, không để lộ thông tin nhạy cảm
        return default ?: ""
    }
}
```

KẾT QUẢ

```
RootDetectionHelper.kt  SignupActivity.kt  com.cx.vulnerablekotlinapp_secure.xml  Client.kt
1  <?xml version='1.0' encoding='utf-8' standalone='yes' ?>
2  <map>
3      <string name="userName">MvH9S1nqmQo3TPfP&#10;;Gau8jz2yks1R9x1KiP54JBJtPAEKZ6h5Ka8=&#10;
4      <string name="userId">XsLPyK60q/HDXYqZ&#10;;G42POv6CNKgA4TBoeC3E2xM=&#10;
5  </map>
6
```

M-1

IMPROPER PLATFORM USAGE (SỬ DỤNG NỀN TẢNG KHÔNG ĐÚNG CÁCH)



PHÂN TÍCH

- Lỗi hỏng **Improper Platform Usage** xuất hiện do việc cấu hình sai các **Content Providers**.
- **Content Providers** là một thành phần quan trọng trong hệ thống Android để chia sẻ dữ liệu giữa các ứng dụng.

PHÂN TÍCH

AccountProvider trong file **AndroidManifest** được khai báo với thuộc tính **exported="true"**, cho phép mọi ứng dụng trên thiết bị truy cập vào.

```
<provider  
    android:name="com.cx.goatlin.AccountProvider"  
    android:authorities="com.cx.goatlin.accounts"  
    android:enabled="true"  
    android:exported="true" />
```

PHÂN TÍCH

Lớp **AccountProvider.kt** cho thấy **Content URI** được định nghĩa như sau:

```
companion object {  
    private val AUTHORITY = "com.cx.goatlin.accounts"  
    private val ACCOUNTS_TABLE = "Accounts"  
    val CONTENT_URI : Uri = Uri.parse( uriString: "content://" + AUTHORITY + "/" +  
        ACCOUNTS_TABLE)  
    private val DATABASE_NAME = "data"  
}  
}
```

PHÂN TÍCH

Kẻ tấn công có thể sử dụng công cụ **adb** để thực hiện truy vấn hoặc chèn dữ liệu trái phép vào **Content Provider** này.

```
D:\Android>adb shell content query --uri content://com.cx.goatlin.accounts/Accounts
Row: 0 id=1, username=admin, password=12345678
Row: 1 id=2, username=, password=
Row: 2 id=3, username=huyho, password=huyho
```

```
D:\Android>adb shell content insert --uri content://com.cx.goatlin.accounts/Accounts --bind username:s:huyho123 --bind password:s:huyho123
```

```
D:\Android>adb shell content query --uri content://com.cx.goatlin.accounts/Accounts
Row: 0 id=1, username=admin, password=12345678
Row: 1 id=2, username=, password=
Row: 2 id=3, username=huyho, password=huyho
Row: 3 id=4, username=huyho123, password=huyho123
```

PHÂN TÍCH

NotesProvider cũng được khai báo với **exported="true"**, dẫn đến nguy cơ rò rỉ dữ liệu từ các ghi chú của người dùng.

```
<provider  
    android:name="com.cx.goatlin.NotesProvider"  
    android:authorities="com.cx.goatlin.notes"  
    android:enabled="true"  
    android:exported="true"></provider>
```

PHÂN TÍCH

Dữ liệu có thể bị truy vấn thông qua **Content URI** được định nghĩa trong lớp **NotesProvider**.

```
D:\Android>adb shell content query --uri content://com.cx.goatlin.notes/Notes
Row: 0 id=1, title=whvw, content=whvw, createdAt=2024-12-11 15:37:00, owner=3
Row: 1 id=2, title=whvw2, content=gijkmn, createdAt=2024-12-11 15:37:24, owner=3
Row: 2 id=3, title=whvw, content=kxb, createdAt=2024-12-11 16:18:56, owner=4
Row: 3 id=4, title=whvw2, content=kxb, createdAt=2024-12-11 16:19:02, owner=4
D:\Android>
```

CẢI TIẾN

Cấu hình lại **Content Providers** bằng cách đặt thuộc tính **exported** về **false** để ngăn chặn truy cập từ các ứng dụng bên ngoài.

```
<provider
    android:name="com.cx.goatlin.AccountProvider"
    android:authorities="com.cx.goatlin.accounts"
    android:enabled="true"
    android:exported="false" />
```

CẢI TIẾN

```
C:\Users\PC>adb shell content insert --uri content://com.c
x.goatlin.accounts/Accounts --bind username:s:huyho --bin
d password:s:huyho
Error while accessing provider:com.cx.goatlin.accounts
java.lang.SecurityException: Permission Denial: opening pr
ovider com.cx.goatlin.AccountProvider from (null) (pid=533
0, uid=2000) that is not exported from UID 10083
    at android.os.Parcel.readException(Parcel.java:194
2)
    at android.os.Parcel.readException(Parcel.java:188
8)
    at android.app.IActivityManager$Stub$Proxy.getCont
entProviderExternal(IActivityManager.java:7137)
    at com.android.commands.content.Content$Command.ex
ecute(Content.java:441)
    at com.android.commands.content.Content.main(Conte
nt.java:664)
    at com.android.internal.os.RuntimeInit.nativeFinis
hInit(Native Method)
    at com.android.internal.os.RuntimeInit.main(Runtim
eInit.java:284)
```


TÁC DỤNG

- Đặt **exported="false"** giúp ngăn chặn các ứng dụng khác truy cập trái phép.
- Loại bỏ nguy cơ rò rỉ thông tin đăng nhập.
- Ngăn chặn việc chèn dữ liệu trái phép, đảm bảo tính toàn vẹn của hệ thống.
- Tăng cường khả năng mở rộng với cấu hình bảo mật chặt chẽ.

KẾT LUẬN

Lỗi hỏng **Improper Platform Usage** trong Goatlin được khắc phục bằng cách:

- Cấu hình lại Content Providers.
- Bổ sung xác thực và mã hóa dữ liệu.

Những cải tiến này giúp:

- Bảo vệ dữ liệu người dùng.
- Hệ thống tuân thủ các tiêu chuẩn bảo mật cao.
- Giảm thiểu các nguy cơ tấn công và rò rỉ dữ liệu.

M-2

INSECURE DATA STORAGE (LƯU TRỮ DỮ LIỆU KHÔNG AN TOÀN)



PHÂN TÍCH

- Hàm **createAccount** trong class **DatabaseHelper** dùng để tạo 1 tài khoản và lưu vào local.
- Hàm này lưu thẳng **plaintext** vào cơ sở dữ liệu mà không thông qua phương thức mã hoá nào để bảo vệ tài khoản user.

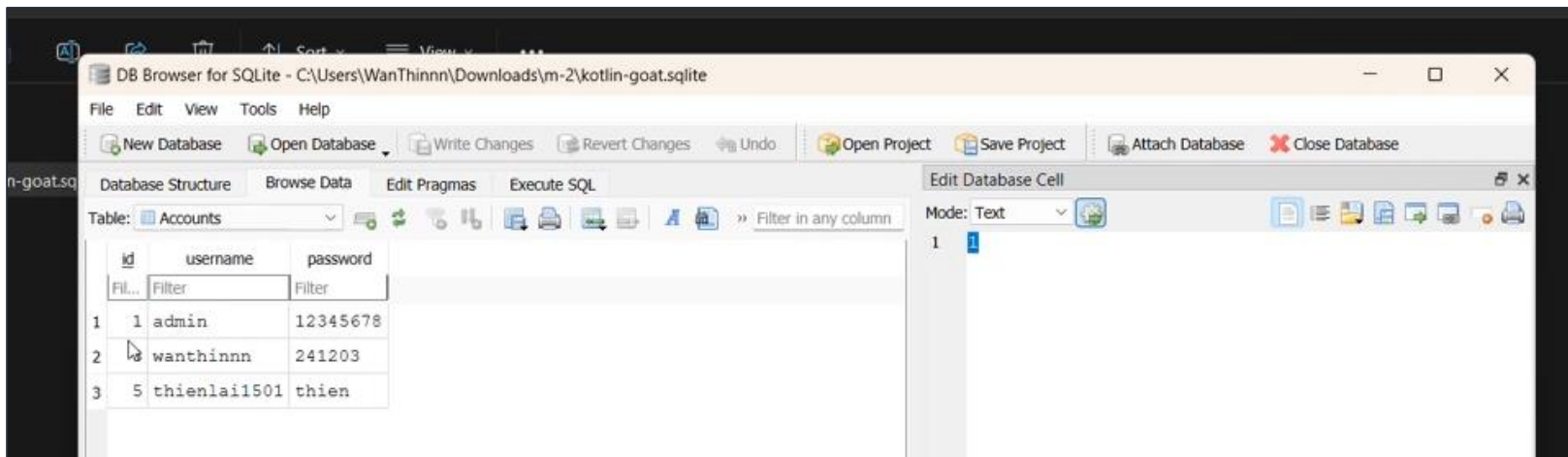
PHÂN TÍCH

```
public fun createAccount(username: String, password: String) : Boolean {  
    val db: SQLiteDatabase = this.writableDatabase  
    val record: ContentValues = ContentValues()  
    var status = true  
  
    record.put("username", username)  
    record.put("password", password)  
  
    try {  
        db.insertOrThrow(TABLE_ACCOUNTS, nullColumnHack: null, record)  
    }  
    catch (e: SQLException) {  
        Log.e(tag: "Database signup", e.toString())  
        status = false  
    }  
    finally {  
        return status  
    }  
}
```

PHÂN TÍCH

Dễ dàng xem được dữ liệu được lưu khi sử dụng **adb** để pull **file database** về máy.

```
C:\Users\WanThinnn>adb -s emulator-5556 pull /data/data/com.cx.vulnerablekotlinapp/databases/data "C:\Users\WanThinnn\Downloads\m-2\kot  
lin-goat.sqlite"  
/data/data/com.cx.vulnerablekotlinapp/databases/data: 1 file pulled. 12.1 MB/s (28672 bytes in 0.002s)
```



CẢI TIẾN

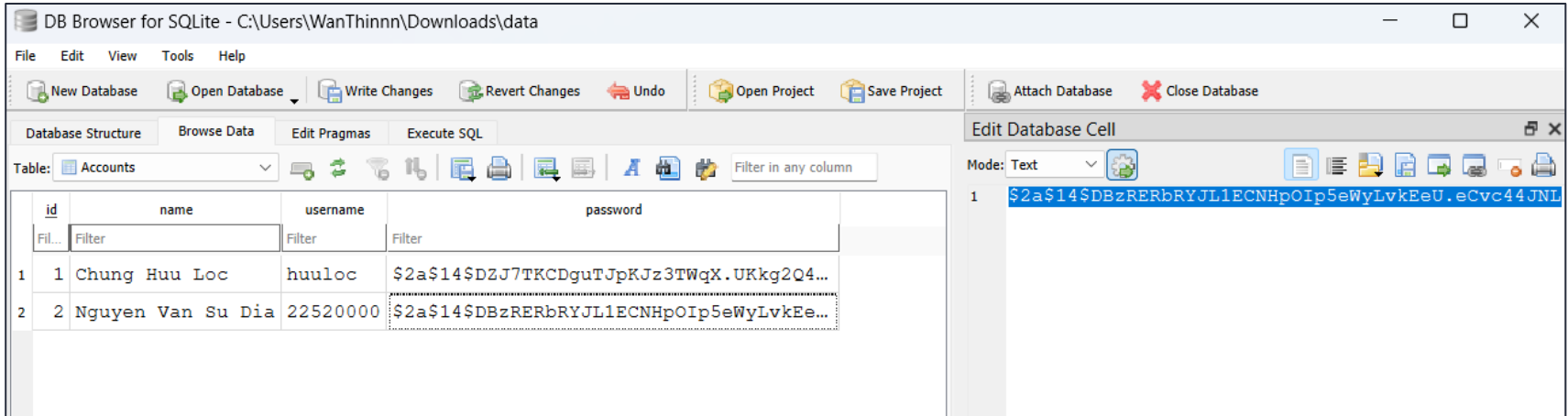
Sử dụng thuật toán **Bcrypt** được **cải tiến với SHA512** và lưu được mật khẩu **dài hơn 72 byte** so với Bcrypt truyền thống.

```
// Mã hóa mật khẩu bằng Bcrypt
fun encryptpw(original: String): String {
    return BCrypt.with(LongPasswordStrategies.hashSha512(BCrypt.Version.VERSION_2A)).hashToString(cost: 14, original.toCharArray()); //allows to honour all pw bytes
}

// Kiểm tra mật khẩu với Bcrypt
fun verifypw(password: String, hashed: String): Boolean {
    return BCrypt.verifyer().verify(password.toCharArray(), hashed).verified
}
```

CẢI TIẾN

Password sẽ được mã hoá và lưu vào cơ sở dữ liệu mỗi khi tạo một tài khoản mới.



The screenshot shows the DB Browser for SQLite application. The main window displays a table named 'Accounts' with the following data:

	id	name	username	password
1	1	Chung Huu Loc	huuloc	\$2a\$14\$DZJ7TKCDguTJpKJz3TWqX.UKkg2Q4...
2	2	Nguyen Van Su Dia	22520000	\$2a\$14\$DBzRERbRYJL1ECNHpOIp5eWyLvKEe...

The 'password' column for the second row is selected, and the 'Edit Database Cell' dialog is open, showing the hashed password: `$2a$14$DBzRERbRYJL1ECNHpOIp5eWyLvKEeU.eCvc44JNL`.

TÁC DỤNG

- **Bảo vệ dữ liệu khi bị tấn công** do các mật khẩu đã được hash.
- **Kháng tấn công brute force** với các thuật toán như Bcrypt được thiết kế giảm tốc độ.
- **Tính an toàn trước rò rỉ** do hashing mật khẩu ngăn chặn khả năng sử dụng lại thông tin đăng nhập trên các hệ thống khác.
- Áp dụng **Bcrypt kết hợp SHA512** đảm bảo **tính tương thích cao** và **khả năng xử lý tốt** với các mật khẩu dài hơn 72 byte.

KẾT LUẬN

- Sử dụng thuật toán **Bcrypt cải tiến** trong ứng dụng cải thiện đáng kể tính bảo mật, bảo vệ người dùng khỏi nguy cơ bị đánh cắp mật khẩu.
- Cần ưu tiên bảo vệ dữ liệu nhạy cảm bằng cách sử dụng các giải pháp mã hóa mạnh mẽ, đặc biệt đối với các thông tin nhạy cảm.

M-3

INSECURE COMMUNICATION (GIAO TIẾP KHÔNG AN TOÀN)



PHÂN TÍCH

- **Insecure Communication** đề cập đến các lỗ hổng hoặc thiếu sót trong cách các ứng dụng di động truyền tải dữ liệu.
- Dẫn tới nguy cơ bị đánh cắp hoặc can thiệp vào thông tin nhạy cảm.

PHÂN TÍCH

Các lỗi hỏng cụ thể như sau:

- **Dữ liệu không được mã hóa:** Truyền dữ liệu qua HTTP thay vì HTTPS.
- **Sử dụng giao thức mã hóa yếu:** Sử dụng SSL/TLS phiên bản lỗi thời hoặc không cấu hình đúng cách, thiếu xác thực chứng chỉ máy chủ.
- **Quản lý phiên không an toàn:** Cookie hoặc token truy cập không được bảo vệ trong khi truyền.

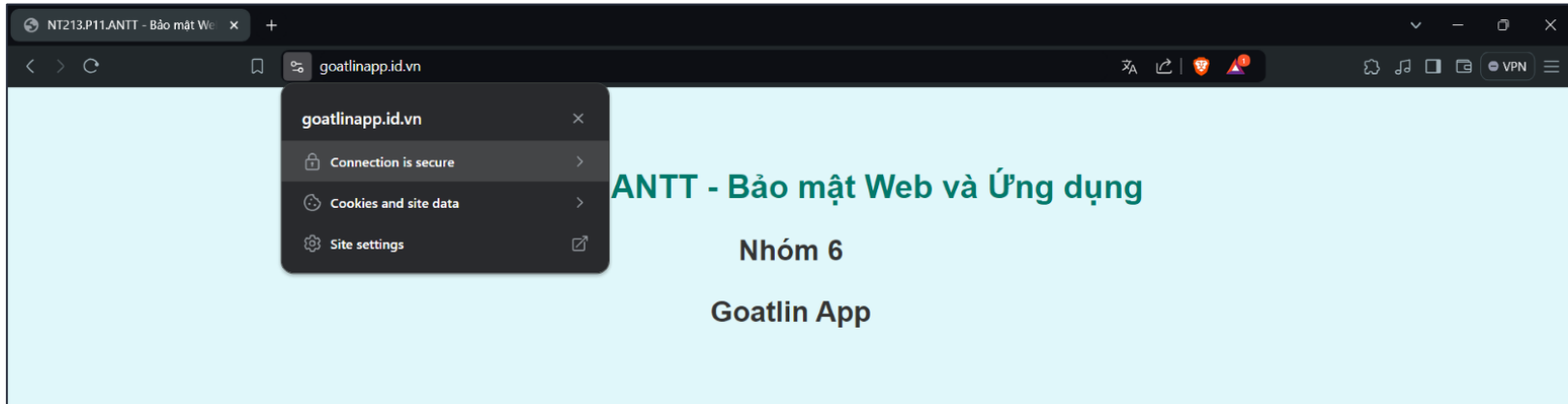
PHÂN TÍCH

Mã nguồn sử dụng giao thức **http://** thay vì **https://**.

```
fun create(): Client {  
    val hostname: String = PreferenceHelper.getString("ip_address", "127.0.0.1")  
    val port: String = PreferenceHelper.getString("port", "8080")  
    val baseUrl: String = "http://${hostname}:${port}"  
  
    val retrofit = Retrofit.Builder()  
        .addCallAdapterFactory(RxJava2CallAdapterFactory.create())  
        .addConverterFactory(GsonConverterFactory.create())  
        .baseUrl(baseUrl)  
        .build()  
  
    return retrofit.create(Client::class.java)  
}
```

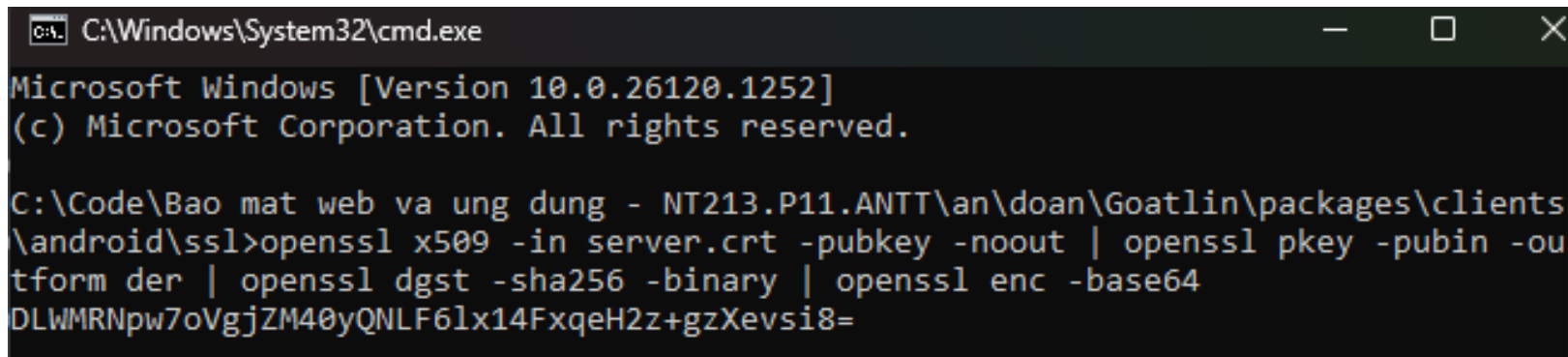
CẢI TIẾN

Tạo **Web Server HTTPS** để thực hiện truyền dữ liệu theo rule được cấu hình sẵn.



CẢI TIẾN

Dùng **server.key** và **server.crt** của trang web để thực hiện sử dụng **Certificate Pinning** giúp ngăn chặn các cuộc tấn công Man-in-the-Middle. Do ứng dụng chỉ chấp nhận chứng chỉ của máy chủ có mã băm **SHA256** đã xác định.



```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26120.1252]
(c) Microsoft Corporation. All rights reserved.

C:\Code\Bao mat web va ung dung - NT213.P11.ANTT\an\doan\Goatlin\packages\clients
\android\ssl>openssl x509 -in server.crt -pubkey -noout | openssl pkey -pubin -ou
tform der | openssl dgst -sha256 -binary | openssl enc -base64
DLWMRNpw7oVgjZM40yQNLf61x14FxqeH2z+gzXevsi8=
```


CẢI TIẾN

Sửa đổi Phương thức tạo của API, sử dụng **OkHttp** để quản lí các yêu cầu mạng.

- **certificatePinner**: cấu hình **Certificate Pinning** để bảo vệ khỏi các cuộc tấn công giả mạo.
- **OkHttpClient**: tạo một OkHttpClient với Certificate Pinning.
- **Retrofit**: cấu hình Retrofit để sử dụng OkHttpClient tùy chỉnh và giao tiếp với máy chủ HTTPS.

```
val client: OkHttpClient = OkHttpClient.Builder()  
    .certificatePinner(certificatePinner)  
    .build()
```

TÁC DỤNG

- Ngăn chặn tấn công Man-in-the-Middle (MitM).
- Bảo mật thông tin nhạy cảm.
- Đáp ứng tiêu chuẩn bảo mật hiện đại.
- Hiệu quả và linh hoạt hơn trong phát triển.

KẾT LUẬN

- Lỗi hỏng **Insecure Communication** dễ dàng được khắc phục bằng cách sử dụng giao thức **HTTPS** và triển khai **Certificate Pinning**.
- Sử dụng **OkHttp** và **Retrofit** giúp bảo mật giao tiếp mạng, cải thiện hiệu suất và tăng tính tiện lợi trong phát triển ứng dụng.

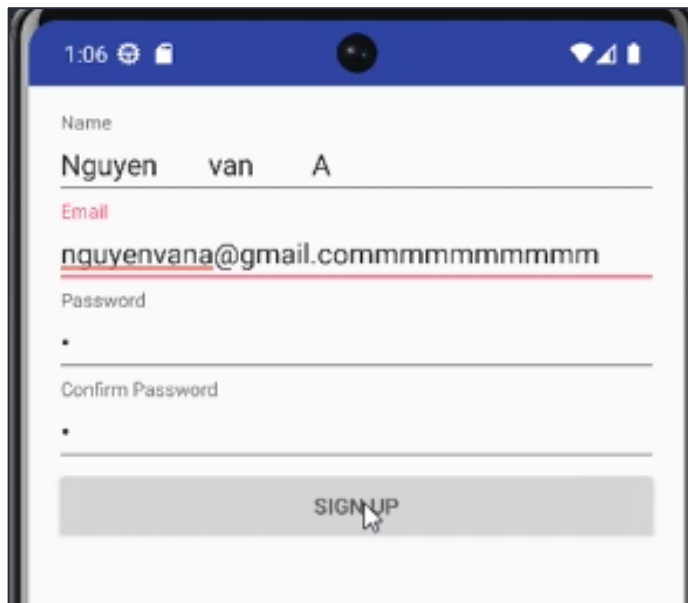
M-4

INSECURE AUTHENTICATION (XÁC THỰC KHÔNG AN TOÀN)



PHÂN TÍCH

Goatlin không có bất kỳ sự ràng buộc nào về mật khẩu, tên người dùng và cả họ tên khi tạo tài khoản.

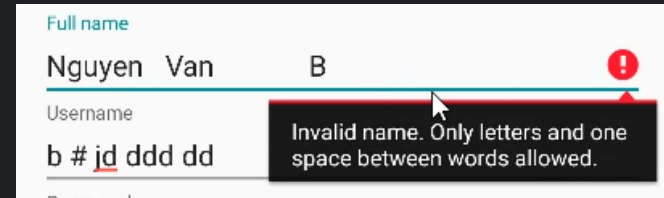


The screenshot shows a mobile application interface for user registration. At the top, there is a status bar with the time 1:06 and various icons. The form consists of four input fields: 'Name' with the text 'Nguyen van A', 'Email' with the text 'nguyenvana@gmail.commmmmmmmmmm', 'Password' with a single dot, and 'Confirm Password' with a single dot. A red underline is visible under the email field. At the bottom of the form is a grey button labeled 'SIGNUP'.

CẢI TIẾN

Sử dụng hàm chuẩn hoá, đảm bảo người dùng không nhập dư thừa ký tự khoảng trắng hoặc ký tự lạ ở phần **Họ tên**.

```
// Kiểm tra và chuẩn hóa tên
if (!CheckStringHelper.validateName(name)) {
    this.name.error = "Invalid name. Only letters and one space between words allowed."
    this.name.requestFocus()
    return
}
else{
    name = CheckStringHelper.capitalizeFirstLetter(name)
}
```

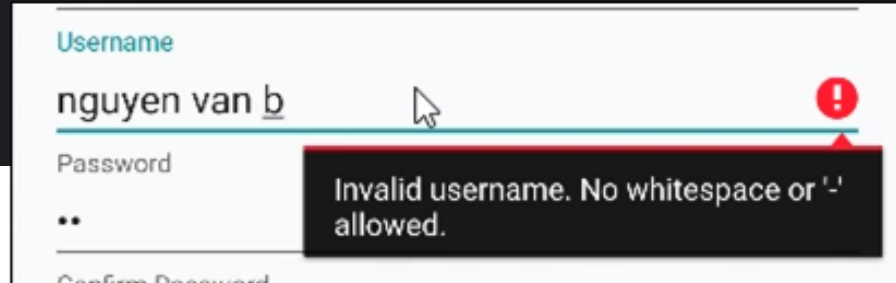


The screenshot shows a web form with three input fields: 'Full name', 'Username', and 'Password'. The 'Full name' field contains the text 'Nguyen Van B' and has a red error message box displayed below it. The error message reads: 'Invalid name. Only letters and one space between words allowed.' The 'Username' field contains 'b #jd ddd dd' and the 'Password' field is empty. A mouse cursor is pointing at the error message box.

CẢI TIẾN

Sử dụng **Username** thay cho **Email**, không sử dụng email vì dữ liệu chỉ lưu ở local, ngoài ra thêm các ràng buộc như: không sử dụng khoảng trắng, không có ký tự lạ,...

```
if (!CheckStringHelper.validateNoWhitespaceOrDash(username)){  
    this.email.error = "Invalid username. No whitespace or '-' allowed."  
    this.email.requestFocus()  
    return  
}
```

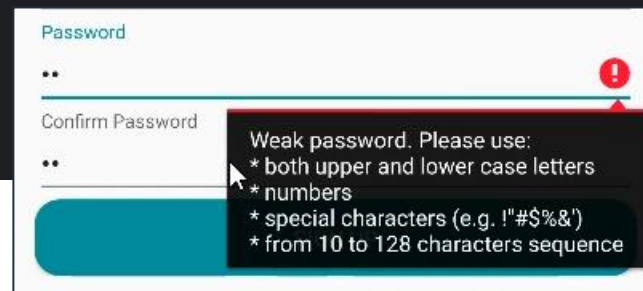


The screenshot shows a login form with three input fields: Username, Password, and Confirm Password. The Username field contains the text "nguyen van b" and has a red error icon (a circle with an exclamation mark) at its right end. A red tooltip box is displayed below the Username field, containing the text "Invalid username. No whitespace or '-' allowed." The Password field contains two dots "••". The Confirm Password field is partially visible at the bottom.

CẢI TIẾN

Sử dụng **Bcrypt** cho mật khẩu và lưu vào CSDL (đề cập ở M-2), đồng thời thêm hàm **PasswordHelper** để kiểm tra độ an toàn của mật khẩu.

```
if (!PasswordHelper.strength(password)) {  
    this.password.error = ""|Weak password. Please use:  
        |* both upper and lower case letters  
        |* numbers  
        |* special characters (e.g. !"#$$%&')  
        |* from 10 to 128 characters sequence""|.trimMargin()  
    this.password.requestFocus()  
    return;  
}
```



The screenshot shows a web form with two input fields: "Password" and "Confirm Password". Both fields contain two asterisks (**). A red exclamation mark icon is visible in the top right corner of the "Password" field. A tooltip is displayed over the "Password" field, containing the following text:

- Weak password. Please use:
- * both upper and lower case letters
- * numbers
- * special characters (e.g. !"#\$\$%&')
- * from 10 to 128 characters sequence

TÁC DỤNG

- Cải thiện chất lượng dữ liệu đầu vào.
- Giảm nguy cơ sử dụng thông tin không xác thực.
- Bảo vệ thông tin đăng nhập.
- Tăng độ tin cậy của ứng dụng.

KẾT LUẬN

Lỗi hỏng **Insecure Authentication** có thể được khắc phục bằng cách:

- Ràng buộc và chuẩn hóa dữ liệu đầu vào để cải thiện tính nhất quán và giảm nguy cơ khai thác thông tin không an toàn.
- Sử dụng thuật toán bảo mật như Bcrypt để hash mật khẩu và tích hợp cơ chế kiểm tra độ mạnh của mật khẩu.

M-5

INSUFFICIENT CRYPTOGRAPHY (MÃ HÓA KHÔNG ĐỦ MẠNH)



PHÂN TÍCH

Class **CryptoHelper** sử dụng thuật toán mã hoá **Caesar**, đây là một thuật toán mật mã học cổ điển và không còn an toàn nữa.

```
class CryptoHelper {
    companion object {
        fun encrypt(original: String): String {
            var encrypted: String = ""

            for (c in original) {
                val ascii: Int = c.toInt()
                val lowerBoundary: Int = if (c.isUpperCase()) 65 else 97

                if (ascii in 65..90 || ascii in 97..122) {
                    encrypted += ((ascii + SHIFT - lowerBoundary) % 26 + lowerBoundary).toChar()
                } else {
                    encrypted += c
                }
            }

            return encrypted
        }

        fun decrypt(encrypted: String): String {
            var original: String = ""

            for (c in encrypted) {
                val ascii: Int = c.toInt()
                val lowerBoundary: Int = if (c.isUpperCase()) 65 else 97

                if (ascii in 65..90 || ascii in 97..122) {
                    original += ((ascii - SHIFT - lowerBoundary) % 26 + lowerBoundary).toChar()
                } else {
                    original += c
                }
            }

            return original
        }
    }
}
```

PHÂN TÍCH

Sử dụng tool online để decrypt dữ liệu tìm được trong file database ta thấy được note của người dùng.

The screenshot shows a web-based tool for decrypting Caesar ciphers. It is divided into three main sections: Ciphertext, Caesar cipher settings, and Plaintext.

- Ciphertext:** Labeled "VIEW" and "Ciphertext". It contains the input text: "Gdb od jkl fxx fxd dss fkxd ila orl".
- Caesar cipher:** Labeled "ENCODE" and "DECODE". It is set to "Caesar cipher". The "SHIFT" is set to "3 a→d". The "ALPHABET" is "abcdefghijklmnopqrstuvwxyz". The "CASE STRATEGY" is "Maintain case". The "FOREIGN CHARS" are "Include Ignore". A status bar at the bottom indicates "→ Decoded 35 chars".
- Plaintext:** Labeled "VIEW" and "Plaintext". It contains the output text: "Day la ghi chu cua app chua fix loi".

CẢI TIẾN

Sử dụng thuật toán mã hóa như **AES-GCM-256** và lưu **khóa AES** vào **Android Keystore**.

The screenshot shows the DB Browser for SQLite application. The main window displays a table named 'Notes' with columns 'id', 'title', and 'content'. The table contains 5 rows of data. The first row is highlighted in blue. The 'Edit Database Cell' dialog is open, showing the selected cell's content: 'X8YZE7EvCESWvB0VjAZgxv+EMUbmZLoQ1GHE...'. The dialog also has a 'Title' field and a 'Your note text' field. The 'Title' field contains 'Xin chào' and the 'Your note text' field contains 'Day la ghi chu cua app da duoc fix, su dung AES-GCM-256'.

	id	title	content
1	1	3LJc0pzGTR8ope4fqXZyb3ubaDw9y7MVkgyL...	UD9fb9tHRapoWpea6qbmNaDB1Iow5s...
2	2	CkiE5EsivReFnM0zTdE2cNkxXsTIpSdM14EI...	62bk0ktzKQWZOxwvK4ZoM9RVqWe9u1...
3	3	F59LLTn7nB5BB+Bh0/...	1r0k514hkUOHohch0EsQz7Ub3Ce09N...
4	5	X8YZE7EvCESWvB0VjAZgxv+EMUbmZLoQ1GHE...	ZC9WVr0GtYZ5GTkvrE3+4ReSuIKhMh...

Edit Note SAVE DELETE

Title

Xin chào

Your note text

Day la ghi chu cua app da duoc fix, su dung AES-GCM-256

CẢI TIẾN

- **Keystore** lưu trữ các khóa bảo mật mà không để lộ chúng cho ứng dụng.
- Dữ liệu được mã hóa bằng **AES** trong chế độ **GCM**
- Phương thức encrypt tạo **IV ngẫu nhiên** cho mỗi lần mã hóa để bảo mật cao hơn. IV được lưu kèm với dữ liệu mã hóa.
- Phương thức decrypt để giải mã dữ liệu được load lên từ CSDL.

TÁC DỤNG

Bảo mật nâng cao

- Mã hóa mạnh mẽ với **AES-GCM-256**
- Đảm bảo tính toàn vẹn với chế độ **GCM**

Quản lý khóa an toàn với **Android Keystore**

- Tách biệt khóa mã hóa
- Cách ly bảo mật
- Chống trộm khóa

TÁC DỤNG

Tăng cường khả năng quản lý dữ liệu

- Tính duy nhất của khóa khi sử dụng **username** để tạo khóa
- Khả năng mở rộng với phương thức **createUserKey**
- Linh hoạt và bảo mật cao với **IV ngẫu nhiên** ở mỗi lần mã hóa

Cải thiện trải nghiệm người dùng

- Giải mã tự động
- Tăng cường bảo mật thông tin cá nhân

KẾT LUẬN

Lỗi hỏng **Insufficient Cryptography** có thể khắc phục bằng cách áp dụng các thuật toán mã hóa mạnh mẽ như **AES-GCM-256** và sử dụng **Android Keystore** để quản lý khóa an toàn.

M-6

INSECURE AUTHORIZATION (ỦY QUYỀN KHÔNG AN TOÀN)



PHÂN TÍCH

Trong ứng dụng **Goatlin**, lỗ hổng **Insecure Authorization** xuất hiện do thiếu kiểm tra quyền hạn (authorization) trong các API endpoint, mặc dù đã có bước xác thực (authentication).

PHÂN TÍCH

- **Authorization không được áp dụng** khi người dùng thực hiện các yêu cầu đến API.
- Bất kỳ người dùng nào sau khi đăng nhập mặc định có thể truy cập và quản lý tài nguyên không thuộc quyền sở hữu của mình.

PHÂN TÍCH

Đây là lỗi hỏng phổ biến khi hệ thống phụ thuộc vào **IDOR** (Insecure Direct Object Reference) hoặc khi giả định rằng endpoint chỉ được truy cập bởi người dùng có quyền hạn đúng.

Các endpoint chỉ kiểm tra xác thực (auth) mà không xác minh quyền sở hữu tài nguyên.

```
router.put('/accounts/:username/notes/:note', auth, async (req, res, next) => {
```

```
router.get('/accounts/:username/notes', auth, async (req, res, next) => {
```

CẢI TIẾN

Bổ sung **middleware ownership** để kiểm tra quyền sở hữu tài nguyên.

```
function ownership (req, res, next) {  
  if (req.params.username !== req.account.username) {  
    res.statusMessage = "Unauthorized"  
    return res.status(403).end();  
  }  
  
  next();  
}  
  
module.exports = ownership
```

CẢI TIẾN

Middleware thực hiện kiểm tra như sau:

- Xác định người dùng hiện tại từ token xác thực.
- So sánh thông tin từ yêu cầu API với thông tin của user đã xác thực.
- Nếu không khớp, yêu cầu bị từ chối với mã lỗi **403 Unauthorized**.

Sau khi bổ sung kiểm tra quyền sở hữu, cập nhật các endpoint API.

```
router.put('/accounts/:username/notes/:note', [auth, ownership], async (req, res, next) => {
```

```
router.get('/accounts/:username/notes', [auth, ownership], async (req, res, next) => {
```


TÁC DỤNG

Cải tiến bảo mật:

- Kết hợp **authentication** và **authorization**
- Giảm rủi ro **IDOR**
- Bảo vệ **API endpoint**

Lợi ích:

- Ngăn chặn truy cập trái phép.
- Tuân thủ nguyên tắc bảo mật tối thiểu.
- Tăng cường khả năng mở rộng.

KẾT LUẬN

Lỗi hỏng **Insecure Authorization** trong Goatlin được khắc phục bằng cách bổ sung **middleware** kiểm tra quyền hạn:

- Cải thiện đáng kể tính bảo mật của ứng dụng.
- Đảm bảo rằng tài nguyên chỉ được truy cập bởi đúng người dùng sở hữu.
- Giảm thiểu nguy cơ lạm dụng và bảo vệ API khỏi các cuộc tấn công tiềm ẩn.

M-8

CODE TAMPERING (SỬA ĐỔI MÃ NGUỒN)



PHÂN TÍCH

- Sau khi ứng dụng di động được phân phối và cài đặt trên thiết bị, mã và dữ liệu sẽ có sẵn tại đó.
- Kẻ tấn công có cơ hội trực tiếp sửa đổi mã, thao túng nội dung bộ nhớ, thay đổi hoặc thay thế API hệ thống hoặc đơn giản là sửa đổi dữ liệu và tài nguyên của ứng dụng

PHÂN TÍCH

Các lỗ hổng cụ thể như sau:

- **Bypass cơ chế bảo mật:** Kẻ tấn công chỉnh sửa mã nguồn hoặc cấu hình ứng dụng để vô hiệu hóa các biện pháp bảo mật.
- **Chèn mã độc (Malicious Code Injection):** Kẻ tấn công tiêm mã độc vào ứng dụng để thực hiện các hành động nguy hại.
- **Can thiệp runtime (Runtime Manipulation):** Thay đổi hành vi của ứng dụng khi đang chạy.
- **Hooking để đánh cắp dữ liệu (Data Interception):** Tấn công vào các API của ứng dụng bằng cách sử dụng các kỹ thuật hooking.

CẢI TIẾN

Thực hiện kiểm tra nếu thiết bị đã bị root qua Lớp **RootDetectionHelper**.

detectDeveloperBuild(): Phát hiện firmware được xây dựng dành cho nhà phát triển hoặc đã chỉnh sửa, thông qua **Build.TAGS**.

Nếu **Build.TAGS** có chứa "**test-keys**", thì thiết bị có khả năng đã bị root.

```
private fun detectDeveloperBuild(): Boolean {  
    return Build.TAGS?.contains("test-keys") == true  
}
```

CẢI TIẾN

detectRootedAPKs(): Phát hiện các ứng dụng liên quan đến root đã được cài đặt.

Dùng **PackageManager** để kiểm tra sự tồn tại của các gói APK phổ biến liên quan đến root.

```
private fun detectRootedAPKs(context: Context): Boolean {  
    val knownRootedAPKs = listOf(  
        "com.noshufou.android.su",  
        "eu.chainfire.supersu",  
        "com.koushikdutta.superuser"  
    )  
    val pm = context.packageManager  
    return knownRootedAPKs.any { packageName →  
        try {  
            pm.getPackageInfo(packageName, 0)  
            true  
        } catch (e: Exception) {  
            false  
        }  
    }  
}
```

CẢI TIẾN

detectForSUBinaries(): Tìm kiếm các **tệp nhị phân (su)** hoặc tệp liên quan đến root trong hệ thống.

Kiểm tra sự tồn tại của các đường dẫn phổ biến liên quan đến root.

```
private fun detectForSUBinaries(): Boolean {  
    val suPaths = listOf(  
        "/system/bin/su",  
        "/system/xbin/su",  
        "/system/app/Superuser.apk",  
        "/system/xbin/daemonsu"  
    )  
    return suPaths.any { path →  
        File(path).exists()  
    }  
}
```


CẢI TIẾN

isDeviceRooted(): Kết hợp ba phương pháp trên để xác định thiết bị có bị root hay không.

Kiểm tra lần lượt các hàm ở trên để kiểm tra thiết bị có bị Root hay không.

```
fun isDeviceRooted(applicationContext: Context?): Boolean {  
    if (applicationContext == null) return false // Trả về false nếu context null  
    return detectDeveloperBuild() || detectRootedAPKs(applicationContext) || detectForSUBinaries()  
}
```

CẢI TIẾN

Phương thức **RootDetectionHelper.isDeviceRooted()** được gọi trong lớp đầu tiên của chương trình để ngăn ứng dụng chạy trên môi trường Root.

```
override fun onCreate(savedInstanceState: Bundle?) {  
  
    super.onCreate(savedInstanceState)  
    PreferenceHelper.init(applicationContext)  
  
    setContentView(R.layout.activity_login)  
    /* Kiểm tra ROOT */  
    if (RootDetectionHelper.isDeviceRooted(applicationContext)) {  
        forceCloseApp()  
    }  
}
```

CẢI TIẾN

Nếu phát hiện môi trường Root, thì người dùng sẽ thấy một hộp thoại và ứng dụng buộc phải đóng bằng hàm **ForceCloseApp()**.

```
private fun forceCloseApp() {  
    val dialog: AlertDialog.Builder = AlertDialog.Builder(this)  
  
    dialog  
        .setMessage("The application can not run on rooted devices")  
        .setCancelable(false)  
        .setPositiveButton("Close Application", DialogInterface.OnClickListener {  
            _, _ → finish()  
        })  
  
    val alert: AlertDialog = dialog.create()  
  
    alert.setTitle("Unsafe Device")  
    alert.show()  
}
```

TÁC DỤNG

Phát hiện thiết bị đã root

- Kiểm tra firmware chỉnh sửa với **Build.TAGS**.
- Phát hiện ứng dụng liên quan đến root như **SuperSU** và **Magisk**.
- Kiểm tra tệp nhị phân liên quan đến root như **su**.
- Kết hợp các phương pháp kiểm tra hạn chế nguy cơ bỏ sót.

Ngăn chặn ứng dụng chạy trên thiết bị root

- Bảo vệ khỏi tấn công **runtime** hoặc bị **hooking** qua các API.
- Ngăn mã độc hại từ môi trường không đáng tin cậy.

KẾT LUẬN

Lỗi hỏng **Code Tampering** có thể được khắc phục bằng cách áp dụng các biện pháp như kiểm tra thiết bị đã root thông qua lớp **RootDetectionHelper**.

M-9

REVERSE ENGINEERING (KỸ THUẬT DỊCH NGƯỢC)



PHÂN TÍCH

- **Reverse Engineering** là quá trình phân tích cấu trúc, chức năng, hoặc hành vi của một ứng dụng để hiểu cách nó hoạt động, mà không có tài liệu hoặc mã nguồn chính thức từ nhà phát triển.
- **Reverse Engineering** có thể được sử dụng bởi kẻ tấn công để tìm kiếm điểm yếu trong ứng dụng, trích xuất thông tin nhạy cảm, hoặc thay đổi ứng dụng cho mục đích độc hại.

PHÂN TÍCH

Giải mã ứng dụng bằng **Apktool** sẽ thấy được mã nguồn **.smali**.

CẢI TIẾN

Bật tính năng **Obfuscation** để làm mờ mã nguồn khi mã nguồn bị **decompile**.


```
build.gradle (:app) x
1  apply plugin: 'com.android.application'
2
3  apply plugin: 'kotlin-android'
4
5  apply plugin: 'kotlin-android-extensions'
6
7  android {
8      compileSdkVersion 26
9      defaultConfig { DefaultConfig it ->
10         applicationId "com.cx.vulnerablekotlinapp"
11         minSdkVersion 26
12         targetSdkVersion 26
13         versionCode 1
14         versionName "1.0"
15         testInstrumentationRunner "android.support.test.runner.AndroidJUnitRunner"
16     }
17     buildTypes { NamedDomainObjectContainer<BuildType> it ->
18         release {
19             minifyEnabled true
20             proguardFiles getDefaultProguardFile('proguard-android.txt'), 'proguard-rules.pro'
21         }
22     }
23     lintOptions { LintOptions it ->
24         abortOnError false
25     }
26 }
27
```

M-10

EXTRANEOUS FUNCTIONALITY (CHỨC NĂNG KHÔNG CẦN THIẾT)



PHÂN TÍCH

Lỗi hỏng **Extraneous Functionality** liên quan đến việc ứng dụng di động có các chức năng hoặc tính năng không cần thiết, chưa được loại bỏ khi ứng dụng được phát hành, hoặc có những tính năng tiềm ẩn được kích hoạt mà người dùng không nhận biết.

PHÂN TÍCH

- Các chức năng dành cho việc phát triển, gỡ lỗi, hoặc thử nghiệm nhưng lại không bị loại bỏ khi ứng dụng chính thức được phát hành.
- Tính năng ẩn, chức năng không sử dụng hoặc quyền truy cập không cần thiết mà người dùng không biết đến.
- Các API, cổng kết nối (ports), hoặc dịch vụ mở không cần thiết, có thể bị khai thác nếu không được bảo vệ đúng cách.

PHÂN TÍCH

Tài khoản và mật khẩu thử nghiệm chưa được loại bỏ.

```
override fun doInBackground(vararg params: Void): Boolean? {  
    if ((mUsername = "Supervisor") and (mPassword = "MySuperSecretPassword123!")){  
        return true  
    }  
}
```

CẢI TIẾN

Loại bỏ tài khoản và mật khẩu thử nghiệm.

3

TỔNG KẾT

TỔNG KẾT

- Hiểu được cách phát triển và bảo mật một ứng dụng Android Kotlin
- Khai thác được các lỗ hổng bảo mật phổ biến trên ứng dụng di động (OWASP Mobile Top 10 2016)
- Vận dụng được các thuật toán mật mã học hiện đại (AES-GCM, Bcrypt).
- Cơ chế phát hiện thiết bị bị root
- Biết cách sử dụng Android Keystore để bảo vệ khóa an toàn.



**THANKS FOR
LISTENING !**